CSE447: Cryptography and Cryptanalysis
Project Report
Project Title: SPHERE (Secure Patient Health Encrypted Record Environment)

Group No: 4, CSE447 Lab Section: 1, Fall 2025	
ID	**Name**
22101584	Mehrab Muhtasim Sayem
22101215	Mustafina Joarder
22101027	Fuad Fardin

Submission Date: 5 January 2026

Table of Contents

1. System Request 3
    Business need: 3
    Business requirements: 3
    Business value: 3
    Special issues or constraints: 3
2. Functional Requirements 3
3. Technology (Framework, Languages) 3
4. Backend Development 3
5. User Interface Design 3
6. Frontend Development 3
7. User Manual 3
8. Performance and Network Analysis 3
9. Github Repo [Public] Link 4
10. Link of Deployed Project 4
11. Individual Contribution 4
12. References 5

1. System Request

Business need:

Healthcare organizations require a secure, encrypted platform to store and manage sensitive patient health records while preventing unauthorized access and data breaches. Current healthcare systems often store data in plaintext or use weak encryption, making them vulnerable to cyberattacks and HIPAA compliance violations. There is a critical need for a cryptographically secure patient health record system that implements end-to-end encryption, multi-factor authentication, and role-based access control to protect patient privacy and ensure data integrity.

Business requirements:

1. Secure User Authentication: Implement login and registration modules with hashed and salted passwords, supporting multiple user roles (patients, doctors, adminis)
2. End-to-End Data Encryption: Encrypt all sensitive user information (username, email, contact details, medical records) before database storage using asymmetric encryption algorithms
3. Multi-Factor Authentication: Enforce two-step verification to prevent unauthorized account access
4. Role-Based Access Control: Define permissions for adminis, doctors, and patients to restrict access.
5. Medical Record Management: Enable doctors to create and manage encrypted patient diagnoses, prescriptions, and medical histories.
6. Appointment Scheduling: Allow patients to book appointments with doctors.
7. Data Integrity Verification: Implement Message Authentication Codes (MAC) using HMAC-SHA256 to detect unauthorized data modifications.
8. Key Management: Provide secure key generation, storage, rotation, and distribution mechanisms for cryptographic operations.
9. Profile Management: Enable users to view and update their encrypted personal information.
10. Secure Session Management: Protect JWT authentication tokens and prevent session hijacking attacks

Business value:

1. Enhanced Patient Privacy: Full encryption of medical records prevents data exposure even during database breaches, building patient trust.
2. Regulatory Compliance: Meets HIPAA, GDPR, and healthcare data protection standards, avoiding costly legal penalties.
3. Reduced Security Risks: From-scratch cryptographic implementations eliminate backdoor vulnerabilities from third-party libraries.
4. Operational Efficiency: Doctors can securely access patient records and create diagnoses digitally, reducing paperwork.
5. Competitive Advantage: Advanced security features differentiate the platform in the healthcare market.

6. Audit Trail: HMAC data integrity verification creates tamper-proof audit logs for compliance requirements.
7. Scalable Security: Multi-level encryption combining RSA and ECC provides future-proof protection against evolving threats
8. Cost Savings: Prevents data breach incidents that average $4.35 million per incident in healthcare industry
9. User Confidence: Two-factor authentication and secure session management increase user adoption rates
10. Research Capability: Demonstrates practical application of advanced cryptography concepts for academic and commercial purposes

Special issues or constraints:

1. Custom Cryptography Requirement: All encryption algorithms (RSA, ECC, SHA-256, HMAC) must be implemented from scratch without using built-in cryptographic libraries, increasing development complexity
2. Asymmetric-Only Encryption: System exclusively uses asymmetric encryption (RSA and ECC); symmetric encryption algorithms are prohibited, which may impact performance for large data encryption
3. Key Size Limitations: RSA 1024-bit and ECC 256-bit key sizes balance security and performance but require periodic review as computing power advances
4. Database Overhead: Storing encrypted data as text (Base64 encoded) increases database storage requirements by approximately 33% compared to plaintext
5. Performance Considerations: Asymmetric encryption operations are computationally expensive, potentially causing latency during high-volume user registrations and data access
6. Key Persistence: RSA and ECC keys must be securely stored in JSON files on the server filesystem, requiring proper file permissions and backup strategies
7. Multi-Level Encryption Complexity: Optional combined RSA+ECC encryption increases processing time and requires careful key management coordination
8. Search Functionality: Encrypted data cannot be directly searched; requires maintaining SHA-256 hash indexes for username and email lookups
9. CORS Configuration: Development environment allows localhost origins, requiring stricter configuration for production deployment
10. Dependency Management: Frontend and backend must coordinate encryption/decryption operations, requiring synchronized key distribution
11. Error Handling: Cryptographic operations may fail due to invalid keys or corrupted data, requiring robust exception handling
12. Scalability: In-memory key storage limits horizontal scaling; future iterations may require distributed key management solutions

2. Functional Requirements

User Authorization and Authentication.

- The system will support the use of distinct registration processes between patients and doctors that have role-specific data fields.
- User passwords will be hashed with random salt into system using the SHA-256 hash algorithm and stored into the database.
- Password strength will be checked by system upon registration (minimum length)
- System will create and authenticate JWT session management tokens at adjustable expiration periods.
- Two-factor authentication (2FA) will be enabled in the system and based on Email OTP with expiration.
- System will make use of the 2FA verification and allow the access to the protected resources.
- System will include forget password service with email authentication.
- System will automatically enable first registered user to have administrator role assigned to him or her.

Data Encryption & Decryption

- Before user personal information (username, email, name, contact number) is stored, the system shall encrypt the information using RSA-1024.
- System will encrypt data (specialization) of doctors with ECC (secp256k1 curve).
- System will use ECC (secp256k1 curve) to encrypt data of patients (age, sex).
- System will automatically de-encrypt data retrieved in database to authorized users.
- System will enable a multi-level encryption using a RSA and ECC encryption together.
- The System shall store encrypted data using Base64 as a text storage format.
- System will manage encrypted error and will give informative error messages.
- System will encrypt records of diagnosis such as symptoms, diagnosis details and prescriptions using RSA and ECC.

Key Management

- RSA key pairs (1024-bit) shall be generated by system as it starts first application.
- System will use secp256k1 parameters to generate ECC key pairs in initial start.
- System will store RSA keys in the form of JSON at rsa_keys.json.
- System will store ECC keys in ecc_keys.json as JSON.
- System will load stored keys into the system during the next application startups.
- System will create 2FA secret keys to individual users on 2FA enrollment.
- System will use environment variables or secure configuration to secure MAC keys.

Confirmation of Data Integrity.

- System will use HMAC-SHA256 to find message authentication codes.
- System will create MAC tags of critical data operation (user creation, record update)
- MAC tags shall be verified by system before data changes are done.
- System will not accept data having invalid or missing MAC tags.
- System will have SHA-256 hash indexes on encrypted username and email fields so that they can be searched.

- During decryption system will verify integrity of data.

Role-Based Access Control (RBAC)

- System will determine three user roles, which include Administrator, Doctor, and Patient.
- The administrators will be provided with special access to the system.
- Physicians will gain access to patient records, develop diagnoses, prescribe drugs and appointments.
- Patients will have access to their records, profiles, appointments and make new appointments.
- System will apply dependency injection to do permission checks on all API endpoints.
- System will throw 403 Forbidden errors in case of unauthorised access.

Medical Record Management

- The physicians will develop encrypted patient records on diagnosis which will contain the symptoms, diagnosis and prescription.
- System will relate diagnoses to a particular doctor-patient relationship.
- Physicians will see all diagnoses they made.
- All the diagnoses that have been developed on behalf of patients shall be viewed by them.
- System will store all diagnosis content (symptoms, diagnosis text, prescriptions) encrypted.
- System will allow diagnosis editing by the doctor who created it.
- System will show diagnosis history in a timely manner with doctor details.
- System will give critical diagnosis sub-pages where all encrypted data is decrypted.

Appointment Management

- The patients will make appointments with the doctors specifying the date, time and the reason.
- System shall ensure that appointment date/time validates against previous dates.
- Encryption of appointment reason field will occur before storage.
- Patients will be able to see all their appointments.
- System will facilitate the tracking of the appointment status (pending, confirmed, completed, cancelled).
- System will show information of doctors (name and specialization) in listing of appointments.

User Profile Management

- Users will be able to see their decrypted profile information.
- Automatic re-encryption of personal information shall be updated by the users.
- Old password verification will be used to change their passwords.
- Physicians will be in charge of their area of specialization.
- Patients will be in charge of their age and sex fields.
- System will have profile update time.
- System shall have user list option which is role filtered (doctors, patients).
- System will show specialization of doctors in doctor listing pages.

- Session management and security management.
- System will create JWT tokens with user ID, role and expiration time.
- System will authenticate JWT signatures over all secure points.
- Expired, or tampered JWT tokens shall be rejected by the system.
- System will invalidate session during log out.

User Interface Requirements.

- System will have responsive web interface that works with desktop and mobile devices.
- System will use the secured paths with authentication.
- System will show easy to understand error messages in case of validation failures.
- System will supply role-based dashboards (admin, doctor, patient).
- System will introduce loading states in the encryption/decryption processes.
- System will have navigation between important pages (home, login, register, profile, appointments, diagnoses).
- System will have a SPHERE branding and logo all the time.

3. Technology (Framework, Languages)

Backend Technologies:

1. Programming Language: Python
2. Web Framework: FastAPI
3. Database ORM: SQLAlchemy
4. Database: SQLite
5. Authentication: Custom JWT implementation
6. Password Hashing: Custom SHA-256 with salt implementation
7. Email Service: SMTP integration for 2FA and password reset
8. Environment Management: python-dotenv for configuration
9. Server: Uvicorn ASGI server

Cryptography Implementations:

1. RSA Encryption: Custom 1024-bit RSA with Miller-Rabin primality testing
2. ECC Encryption: Custom Elliptic Curve Cryptography using secp256k1 curve
3. SHA-256: Custom Secure Hash Algorithm implementation per FIPS 180-4
4. HMAC: Custom Hash-based Message Authentication Code per RFC 2104
5. Key Management: Custom key generation, storage, and loading system
6. Random Number Generation: Python's `random` and `secrets` modules for cryptographic randomness

Frontend Technologies:

4. Programming Language: TypeScript
5. Framework: React

6. Build Tool: Vite
7. Styling: Tailwind CSS
8. HTTP Client: Axios for API communication
9. State Management: React hooks (useState, useEffect) and context API
10. Form Handling: Controlled components with validation
11. Development Server: Vite dev server (port 5173)

12. Backend Development

Code Snippet 1: RSA Encryption Implementation from Scratch

```
class RSA:
    """RSA Encryption/Decryption implementation from scratch"""

    def __init__(self, key_size: int = 1024):
        self.key_size = key_size
        self.public_key = None
        self.private_key = None

    @staticmethod
    def is_prime(n: int, k: int = 5) -> bool:
        """Miller-Rabin primality test"""
        if n < 2:
            return False
        if n == 2 or n == 3:
            return True
        if n % 2 == 0:
            return False

        # Write n-1 as 2^r * d
        r, d = 0, n - 1
        while d % 2 == 0:
            r += 1
            d //= 2

        # Witness loop
        for _ in range(k):
            a = random.randrange(2, n - 1)
            x = pow(a, d, n)

            if x == 1 or x == n - 1:
                continue

            for _ in range(r - 1):
                x = pow(x, 2, n)
                if x == n - 1:
                    break
            else:
                return False

        return True

    @staticmethod
    def generate_prime(bits: int) -> int:
        """Generate a prime number with specified bit length"""
        while True:
            num = random.getrandbits(bits)
```

```python
            # Set MSB and LSB to ensure odd number of correct bit length
            num |= (1 << bits - 1) | 1
            if RSA.is_prime(num):
                return num

    def generate_keypair(self) -> Tuple[Tuple[int, int], Tuple[int, int]]:
        """
        Generate RSA key pair
        Returns: ((e, n), (d, n)) where (e,n) is public key and (d,n) is private key
        """
        # Generate two distinct primes
        p = self.generate_prime(self.key_size // 2)
        q = self.generate_prime(self.key_size // 2)

        while p == q:
            q = self.generate_prime(self.key_size // 2)

        # Calculate n and phi(n)
        n = p * q
        phi = (p - 1) * (q - 1)

        # Choose e (commonly 65537)
        e = 65537

        # Ensure e and phi are coprime
        while math.gcd(e, phi) != 1:
            e = random.randrange(2, phi)

        # Calculate d (private exponent)
        d = self.mod_inverse(e, phi)

        self.public_key = (e, n)
        self.private_key = (d, n)

        return self.public_key, self.private_key

    @staticmethod
    def encrypt_block(plaintext_block: int, public_key: Tuple[int, int]) -> int:
        """Encrypt a single block using public key"""
        e, n = public_key
        return pow(plaintext_block, e, n)

    @staticmethod
    def decrypt_block(ciphertext_block: int, private_key: Tuple[int, int]) -> int:
        """Decrypt a single block using private key"""
        d, n = private_key
        return pow(ciphertext_block, d, n)
```

Description: This snippet demonstrates the complete RSA encryption algorithm implemented from scratch without using any built-in cryptographic libraries. It includes Miller-Rabin primality testing for generating large prime numbers, Extended Euclidean Algorithm for calculating modular multiplicative inverse, and the core RSA encryption/decryption operations. The implementation generates 1024-bit key pairs and performs modular exponentiation for encrypting/decrypting data blocks, fulfilling the requirement of asymmetric encryption implemented from scratch.

Code Snippet 2: SHA-256 and HMAC Implementation from Scratch

```python
class SHA256:
    """
    SHA-256 Hash Function Implementation from Scratch
    Follows FIPS 180-4 specification
```

```python
    """

    # Initial hash values (first 32 bits of fractional parts of square roots of first 8 primes)
    H0 = [
        0x6a09e667, 0xbb67ae85, 0x3c6ef372, 0xa54ff53a,
        0x510e527f, 0x9b05688c, 0x1f83d9ab, 0x5be0cd19
    ]

    # Round constants (first 32 bits of fractional parts of cube roots of first 64 primes)
    K = [
        0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5,
        # ... (64 constants total)
    ]

    def _process_block(self, block: bytes, h: list) -> list:
        """Process a single 512-bit block"""
        # Prepare message schedule (64 words)
        w = list(struct.unpack('>16I', block)) + [0] * 48

        for i in range(16, 64):
            s0 = (self._right_rotate(w[i-15], 7) ^
                  self._right_rotate(w[i-15], 18) ^
                  (w[i-15] >> 3))
            s1 = (self._right_rotate(w[i-2], 17) ^
                  self._right_rotate(w[i-2], 19) ^
                  (w[i-2] >> 10))
            w[i] = (w[i-16] + s0 + w[i-7] + s1) & 0xFFFFFFFF

        # Initialize working variables
        a, b, c, d, e, f, g, hh = h

        # Main compression loop
        for i in range(64):
            S1 = (self._right_rotate(e, 6) ^
                  self._right_rotate(e, 11) ^
                  self._right_rotate(e, 25))
            ch = (e & f) ^ ((~e) & g)
            temp1 = (hh + S1 + ch + self.K[i] + w[i]) & 0xFFFFFFFF

            S0 = (self._right_rotate(a, 2) ^
                  self._right_rotate(a, 13) ^
                  self._right_rotate(a, 22))
            maj = (a & b) ^ (a & c) ^ (b & c)
            temp2 = (S0 + maj) & 0xFFFFFFFF

            hh = g
            g = f
            f = e
            e = (d + temp1) & 0xFFFFFFFF
            d = c
            c = b
            b = a
            a = (temp1 + temp2) & 0xFFFFFFFF

        return [
            (h[0] + a) & 0xFFFFFFFF,
            (h[1] + b) & 0xFFFFFFFF,
            (h[2] + c) & 0xFFFFFFFF,
            (h[3] + d) & 0xFFFFFFFF,
            (h[4] + e) & 0xFFFFFFFF,
            (h[5] + f) & 0xFFFFFFFF,
            (h[6] + g) & 0xFFFFFFFF,
            (h[7] + hh) & 0xFFFFFFFF
```

```python
        ]

class HMAC:
    """
    HMAC Implementation from Scratch
    HMAC(K, m) = H((K' ⊕ opad) || H((K' ⊕ ipad) || m))
    """

    def __init__(self, key: Union[str, bytes], hash_func=None):
        self.hash_func = hash_func or SHA256()
        self.block_size = 64  # SHA-256 block size
        if isinstance(key, str):
            key = key.encode('utf-8')
        # Key derivation if key longer than block size
        if len(key) > self.block_size:
            key = self.hash_func.hash(key)
        self.key = key.ljust(self.block_size, b'\x00')

    def compute(self, message: Union[str, bytes]) -> bytes:
        """Compute HMAC"""
        if isinstance(message, str):
            message = message.encode('utf-8')

        # Inner padding (ipad = 0x36 repeated)
        ipad = bytes([0x36] * self.block_size)
        # Outer padding (opad = 0x5c repeated)
        opad = bytes([0x5c] * self.block_size)

        # XOR key with pads
        key_ipad = bytes([a ^ b for a, b in zip(self.key, ipad)])
        key_opad = bytes([a ^ b for a, b in zip(self.key, opad)])

        # HMAC = H(K ⊕ opad || H(K ⊕ ipad || message))
        inner_hash = self.hash_func.hash(key_ipad + message)
        return self.hash_func.hash(key_opad + inner_hash)
```

Description: This snippet showcases the SHA-256 hashing algorithm implemented completely from scratch following FIPS 180-4 specification, along with HMAC (Hash-based Message Authentication Code) per RFC 2104. The implementation includes message padding, block processing with 64 rounds of compression using bitwise operations (XOR, AND, rotation), and proper initialization vectors. This fulfills the requirement for data integrity verification using Message Authentication Codes to detect unauthorized modifications, all implemented without built-in cryptographic functions.

Code Snippet 3: ECC (Elliptic Curve Cryptography) Implementation

```python
class Point:
    """Represents a point on an elliptic curve"""

    def __init__(self, x: Optional[int], y: Optional[int], curve: 'EllipticCurve'):
        self.x = x
        self.y = y
        self.curve = curve
        self.is_infinity = (x is None and y is None)

    def __add__(self, other: 'Point') -> 'Point':
        """Point addition on elliptic curve"""
        if self.is_infinity:
            return other
        if other.is_infinity:
            return self

        p = self.curve.p
```

```python
        # Handle point doubling
        if self.x == other.x:
            if self.y == other.y:
                return self.double()
            else:
                return Point(None, None, self.curve)

        # Point addition: slope = (y2 - y1) / (x2 - x1) mod p
        slope = ((other.y - self.y) * self.curve.mod_inverse(other.x - self.x, p)) % p
        x3 = (slope * slope - self.x - other.x) % p
        y3 = (slope * (self.x - x3) - self.y) % p

        return Point(x3, y3, self.curve)

    def double(self) -> 'Point':
        """Point doubling on elliptic curve"""
        if self.is_infinity:
            return self

        p = self.curve.p
        a = self.curve.a

        # slope = (3x^2 + a) / (2y) mod p
        slope = ((3 * self.x * self.x + a) *
                 self.curve.mod_inverse(2 * self.y, p)) % p
        x3 = (slope * slope - 2 * self.x) % p
        y3 = (slope * (self.x - x3) - self.y) % p

        return Point(x3, y3, self.curve)

    def scalar_multiply(self, k: int) -> 'Point':
        """Scalar multiplication using double-and-add algorithm"""
        result = Point(None, None, self.curve)  # Point at infinity
        addend = self

        while k:
            if k & 1:
                result = result + addend
            addend = addend.double()
            k >>= 1

        return result

class ECC:
    """ECC Encryption/Decryption using secp256k1 curve"""

    def __init__(self):
        # secp256k1: y^2 = x^3 + 7 (mod p)
        self.curve = EllipticCurve(
            a=0, b=7,
            p=0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEFFFFFC2F,
            G=(0x79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798,
               0x483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8),
            n=0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
        )

    def generate_keypair(self) -> Tuple[Point, int]:
        """Generate ECC key pair"""
        private_key = random.randint(1, self.curve.n - 1)
        public_key = self.curve.G.scalar_multiply(private_key)
        self.private_key = private_key
        self.public_key = public_key
```

```python
        return public_key, private_key

    def encrypt(self, plaintext: str, public_key: Point) -> str:
        """ElGamal-style ECC encryption"""
        from .mac import SHA256
        sha256 = SHA256()

        plaintext_bytes = plaintext.encode('utf-8')
        plaintext_hash = sha256.hash(plaintext_bytes)
        m = int.from_bytes(plaintext_hash[:16], byteorder='big')

        k = random.randint(1, self.curve.n - 1)
        C1 = self.curve.G.scalar_multiply(k)  # k * G
        shared = public_key.scalar_multiply(k)  # k * PublicKey
        C2 = (m + shared.x) % self.curve.n

        return f"{C1.x},{C1.y},{C2:064x},{plaintext}"
```

Description: This snippet implements Elliptic Curve Cryptography using the secp256k1 curve (same as Bitcoin) completely from scratch. It includes point addition and doubling operations on the elliptic curve, scalar multiplication for key generation, and ElGamal-style encryption/decryption. The implementation handles the mathematical operations required for ECC including modular inverse calculation using Extended Euclidean Algorithm. This demonstrates the second asymmetric encryption algorithm required by the project, enabling multi-level encryption when combined with RSA.

Code Snippet 4: Automatic Data Encryption in Database Models

```python
class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)

    # Encrypted fields using RSA
    username_encrypted = Column(Text, nullable=False)
    email_encrypted = Column(Text, nullable=False)
    name_encrypted = Column(Text, nullable=False)
    contact_no_encrypted = Column(Text, nullable=True)

    # Hash indexes for searching (SHA256 hash)
    username_hash = Column(String(64), unique=True, index=True)
    email_hash = Column(String(64), unique=True, index=True)

    # Password (already hashed with SHA256+salt)
    hashed_password = Column(String(255), nullable=False)

    # ECC encrypted role-specific fields
    specialization_encrypted = Column(Text, nullable=True)  # Doctor
    age_encrypted = Column(Text, nullable=True)  # Patient
    sex_encrypted = Column(Text, nullable=True)  # Patient

    # RSA instance with persistent key management
    @classmethod
    def get_rsa_instance(cls):
        """Get or create RSA instance with persistent keys"""
        if cls._rsa_instance is None:
            cls._rsa_instance = RSA(key_size=1024)
            # Load existing keys or generate new ones
            if not cls._load_rsa_keys(cls._rsa_instance):
                print(" Generating new RSA keys...")
                cls._rsa_instance.generate_keypair()
```

```python
            cls._save_rsa_keys(cls._rsa_instance)
        return cls._rsa_instance

    # Automatic encryption/decryption properties
    @property
    def username(self):
        """Decrypt username using RSA"""
        if self.username_encrypted:
            try:
                rsa = self.get_rsa_instance()
                return rsa.decrypt(self.username_encrypted, rsa.private_key)
            except Exception as e:
                print(f"Error decrypting username: {e}")
                return None
        return None

    @username.setter
    def username(self, value):
        """Encrypt username using RSA and create hash index"""
        if value:
            try:
                rsa = self.get_rsa_instance()
                self.username_encrypted = rsa.encrypt(value, rsa.public_key)
                self.username_hash = _sha256.hash_hex(value)  # For searching
            except Exception as e:
                print(f"Error encrypting username: {e}")

    @property
    def email(self):
        """Decrypt email using RSA"""
        if self.email_encrypted:
            rsa = self.get_rsa_instance()
            return rsa.decrypt(self.email_encrypted, rsa.private_key)
        return None

    @email.setter
    def email(self, value):
        """Encrypt email using RSA and create hash index"""
        if value:
            rsa = self.get_rsa_instance()
            self.email_encrypted = rsa.encrypt(value, rsa.public_key)
            self.email_hash = _sha256.hash_hex(value)

    # ECC encryption for role-specific fields
    @property
    def specialization(self):
        """Decrypt specialization using ECC"""
        if self.specialization_encrypted:
            ecc = self.get_ecc_instance()
            return ecc.decrypt(self.specialization_encrypted, ecc.private_key)
        return None

    @specialization.setter
    def specialization(self, value):
        """Encrypt specialization using ECC"""
        if value:
            ecc = self.get_ecc_instance()
            self.specialization_encrypted = ecc.encrypt(value, ecc.public_key)
```

Description: This snippet demonstrates how user data is automatically encrypted before storage and decrypted upon retrieval using Python properties (getters/setters). The User model uses RSA encryption for sensitive personal information (username, email, name,

contact) and ECC encryption for role-specific fields (doctor specialization, patient age/sex). Hash indexes using SHA-256 are maintained for encrypted fields to enable database searching without exposing plaintext. This implements the requirement that all user information must be encrypted before storage and decrypted upon retrieval, with key persistence across application restarts.

Code Snippet 5: Password Hashing with Salt & Two-Factor Authentication

```python
# Password Hashing Module
class PasswordManager:
    """Handles password hashing with custom SHA256 and salt"""

    @staticmethod
    def generate_salt(length: int = 32) -> str:
        """Generate cryptographic salt"""
        return secrets.token_hex(length)

    @staticmethod
    def hash_password(password: str) -> str:
        """Hash password with SHA256 and salt"""
        salt = PasswordManager.generate_salt()
        sha256 = SHA256()  # Custom SHA256 implementation
        password_hash = sha256.hash_hex(salt + password)
        return f"{salt}${password_hash}"

    @staticmethod
    def verify_password(plain_password: str, hashed_password: str) -> bool:
        """Verify password using constant-time comparison"""
        try:
            salt, stored_hash = hashed_password.split('$')
            sha256 = SHA256()
            computed_hash = sha256.hash_hex(salt + plain_password)

            # Constant-time comparison prevents timing attacks
            if len(computed_hash) != len(stored_hash):
                return False
            result = 0
            for a, b in zip(computed_hash, stored_hash):
                result |= ord(a) ^ ord(b)
            return result == 0
        except Exception:
            return False

# Authentication Router with 2FA
@router.post("/register/patient", response_model=UserResponse)
async def register_patient(data: PatientRegister, db: Session = Depends(get_db)):
    """Register new patient with encrypted data"""

    # Validate passwords match
    if data.password != data.confirm_password:
        raise HTTPException(status_code=400, detail="Passwords do not match")

    # Check for existing users using hash indexes
    if db.query(User).filter(User.email == data.email).first():
        raise HTTPException(status_code=400, detail="Email already registered")

    # Hash password with salt
    hashed_password = password_manager.hash_password(data.password)

    # Create user with automatic encryption via properties
    user = User(
        username=data.username,      # RSA encrypted
```

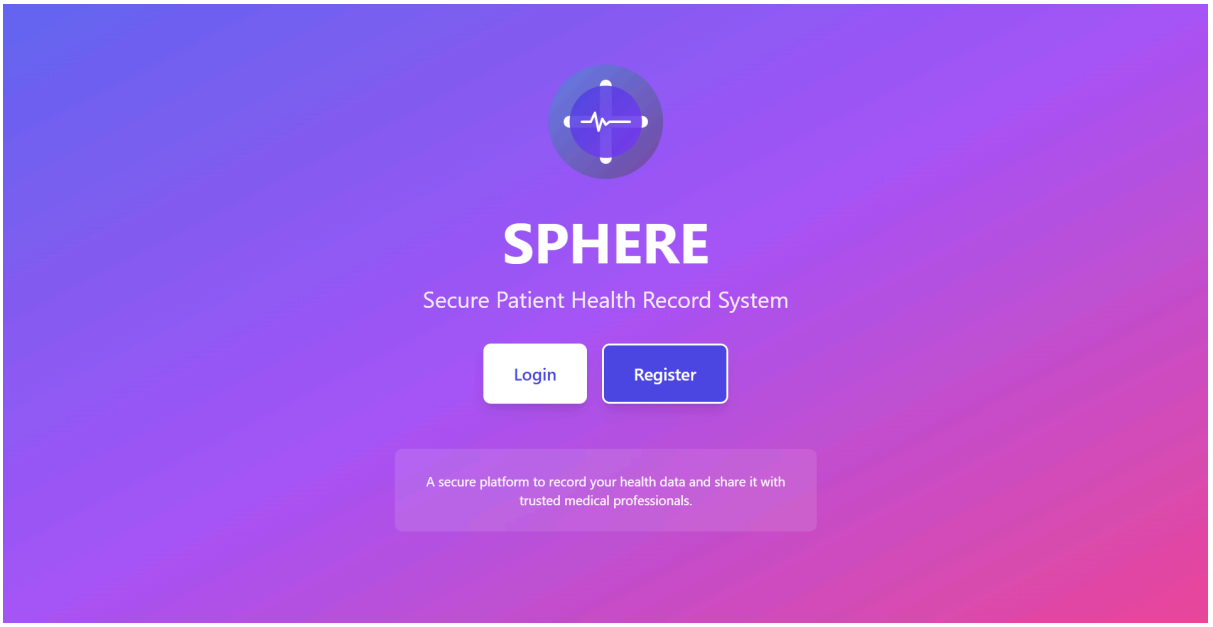
```
        email=data.email,            # RSA encrypted
        name=data.name,              # RSA encrypted
        hashed_password=hashed_password,
        age=data.age,                # ECC encrypted
        sex=data.sex,                # ECC encrypted
        contact_no=data.contact_no,  # RSA encrypted
        role="patient",
        two_factor_enabled=True      # Enable 2FA
    )

    db.add(user)
    db.commit()
    db.refresh(user)
    return user

@router.post("/login", response_model=LoginResponse)
async def login(data: UserLogin, db: Session = Depends(get_db)):
    """Login with two-factor authentication"""

    # Find user by email hash
    sha256 = SHA256()
    email_hash = sha256.hash_hex(data.email)
    user = db.query(User).filter(User.email_hash == email_hash).first()

    if not user:
        raise HTTPException(status_code=401, detail="Invalid credentials")

    # Verify password using custom SHA256 implementation
    if not password_manager.verify_password(data.password, user.hashed_password):
        raise HTTPException(status_code=401, detail="Invalid credentials")

    # Check if 2FA is required
    if user.two_factor_enabled:
        two_fa_code = two_fa.generate_code()
        # Send 2FA code via email
        email_service.send_2fa_code(user.email, two_fa_code)

        # Create temporary token with 2FA code
        temp_token = jwt_manager.create_temp_token(user.id, two_fa_code)

        return {
            "requires_2fa": True,
            "temp_token": temp_token,
            "message": "2FA code sent to email"
        }

    # If no 2FA, return access token
    access_token = jwt_manager.create_access_token(user.id, user.role)
    return {"access_token": access_token, "requires_2fa": False}

@router.post("/verify-2fa", response_model=TokenResponse)
async def verify_2fa(temp_token: str, code: str, db: Session = Depends(get_db)):
    """Verify 2FA code and grant access"""

    # Decode temporary token
    payload = jwt_manager.decode_token(temp_token)
    user_id = payload.get("user_id")
    stored_code = payload.get("2fa_code")

    # Verify 2FA code
    if not two_fa.verify_code(code, stored_code):
        raise HTTPException(status_code=401, detail="Invalid 2FA code")
```

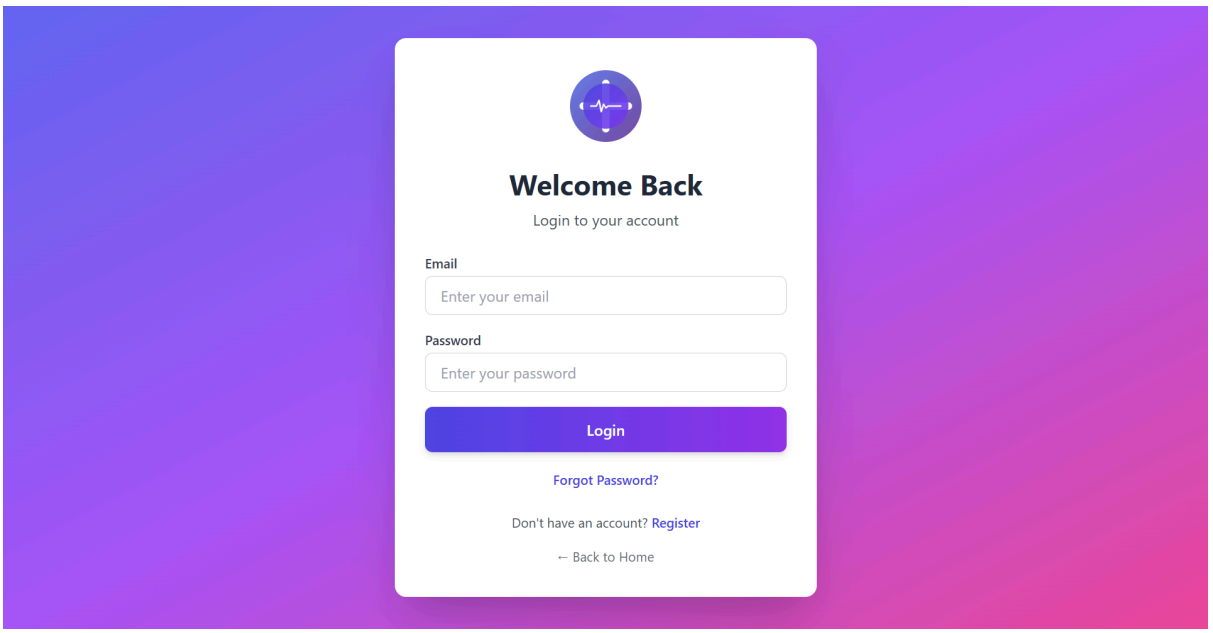
```
    user = db.query(User).filter(User.id == user_id).first()

    # Create permanent access token
    access_token = jwt_manager.create_access_token(user.id, user.role)

    return {"access_token": access_token, "token_type": "bearer"}
```

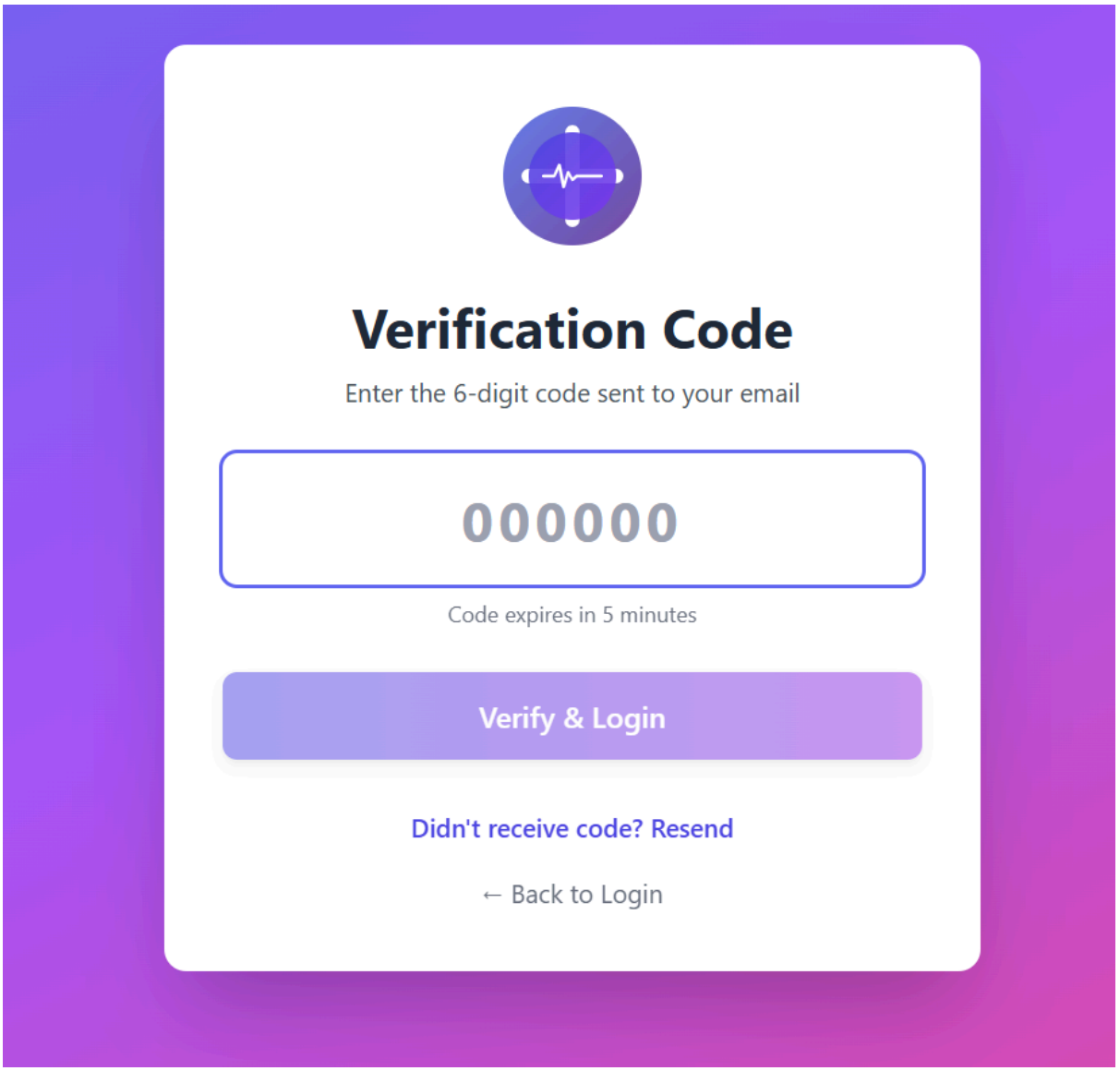
Description: This snippet demonstrates secure password management using custom SHA-256 implementation with cryptographic salts, and two-factor authentication verification during login. Passwords are hashed with a 32-byte random salt before storage, and verification uses constant-time comparison to prevent timing attacks. The login process enforces two-step authentication where users must first validate their primary credentials (username/password) and then provide a second factor (2FA code) before receiving access tokens. This fulfills the requirements for hashed and salted passwords and multi-factor authentication.

13. User Interface Design

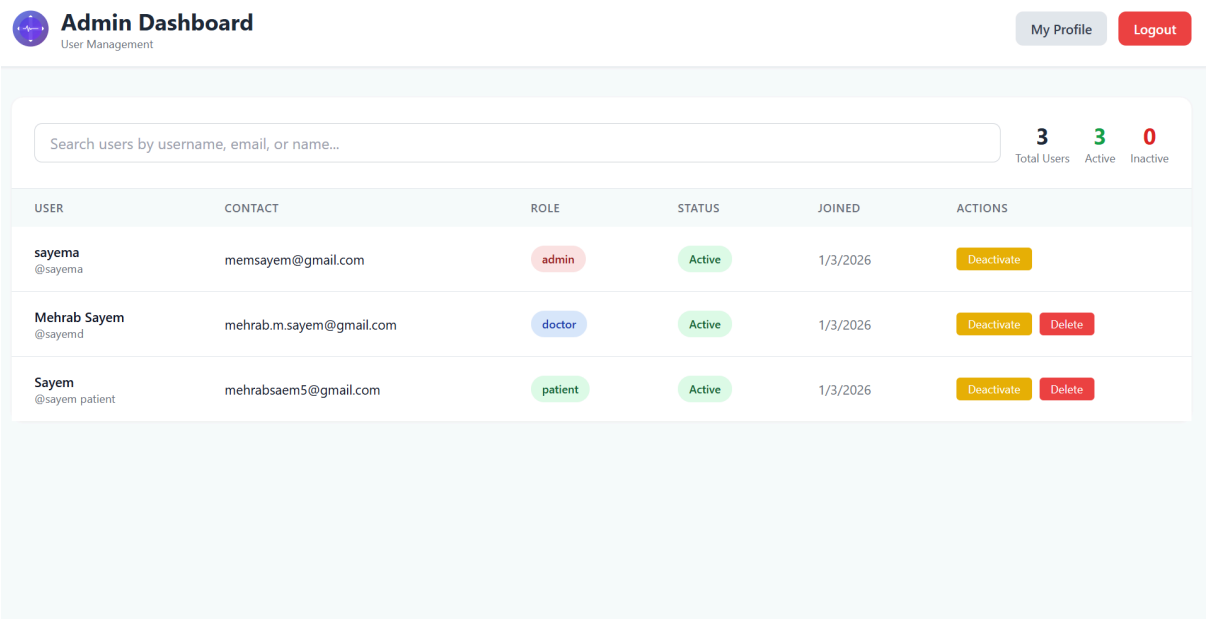
1. Homepage and Login



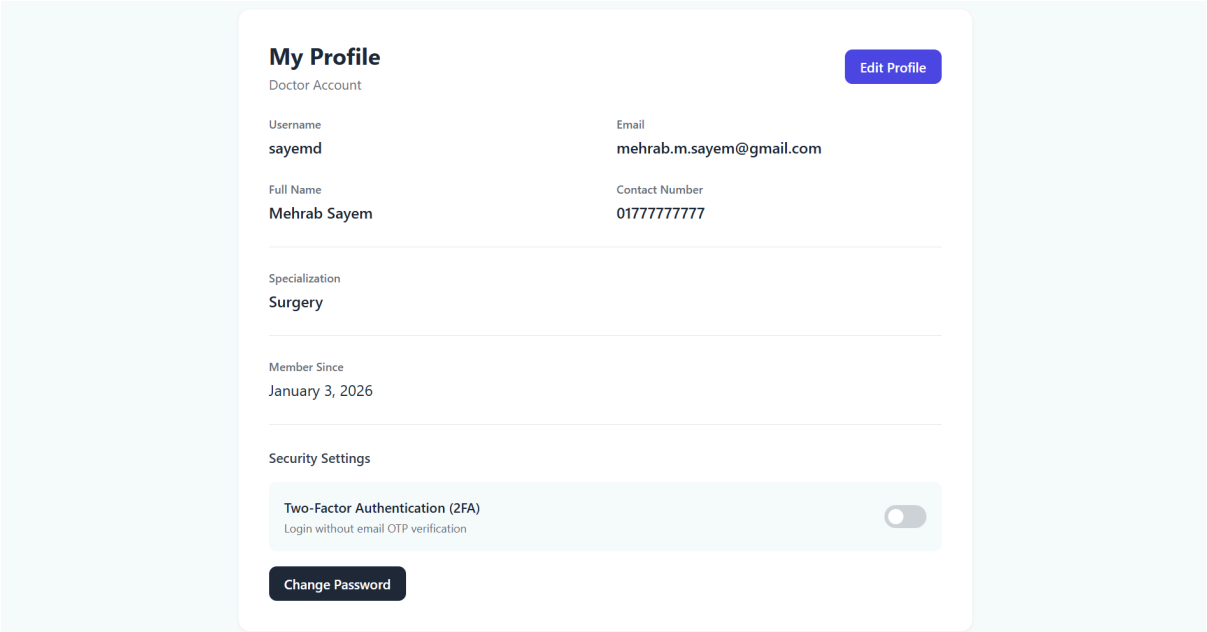


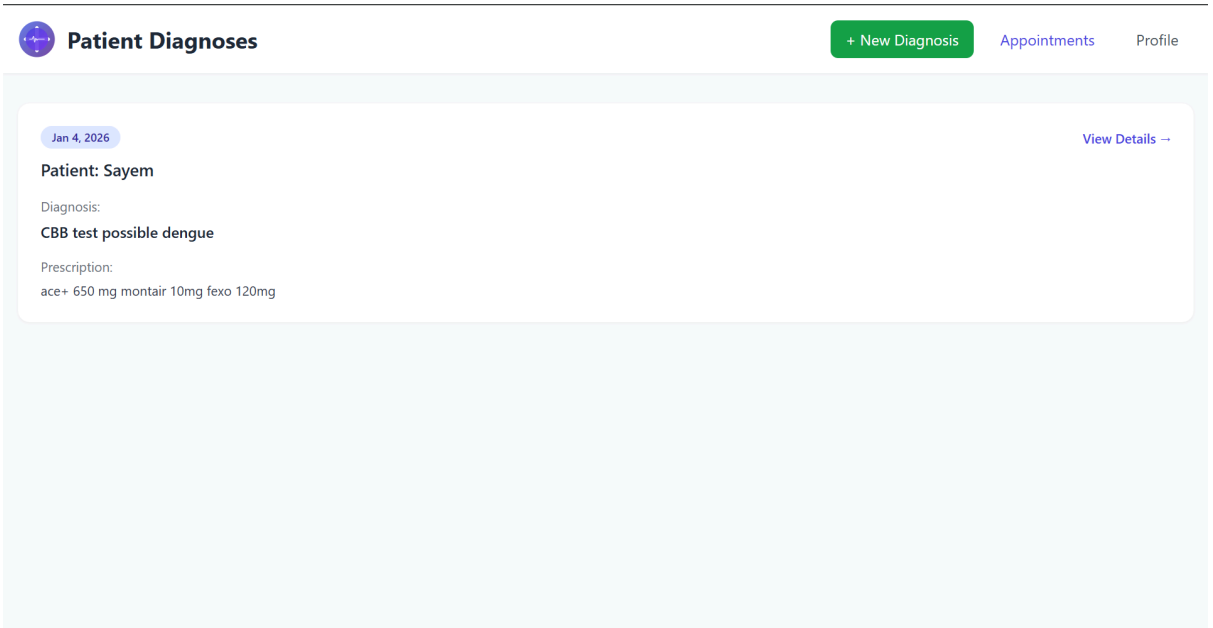


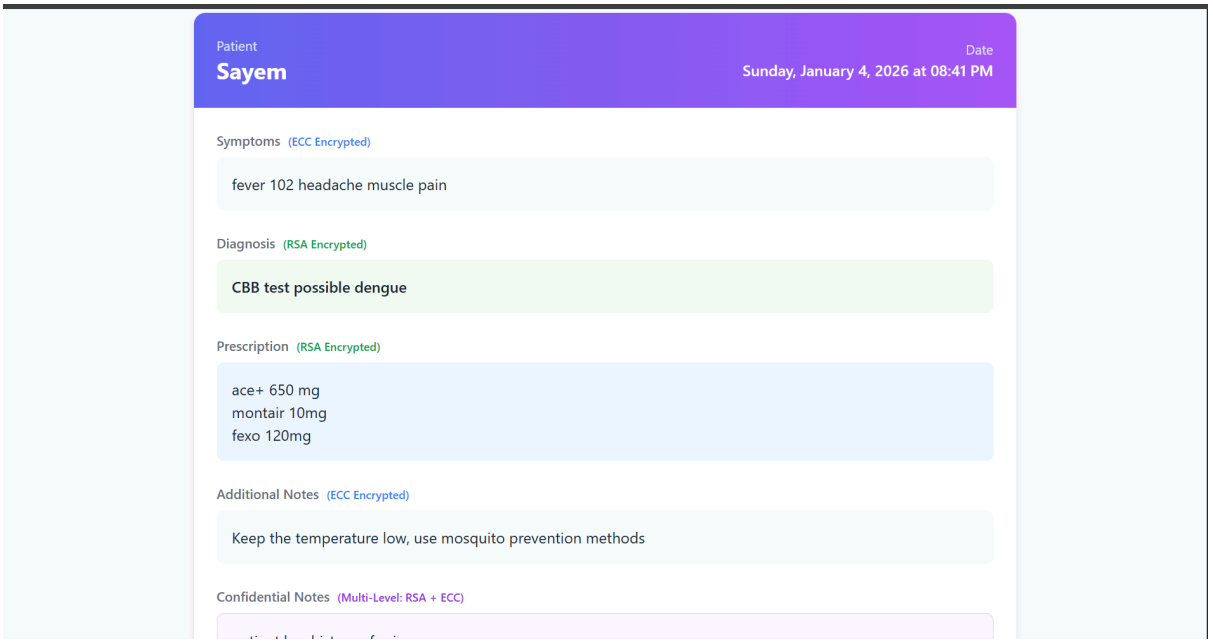
2. Admin Control Interface:

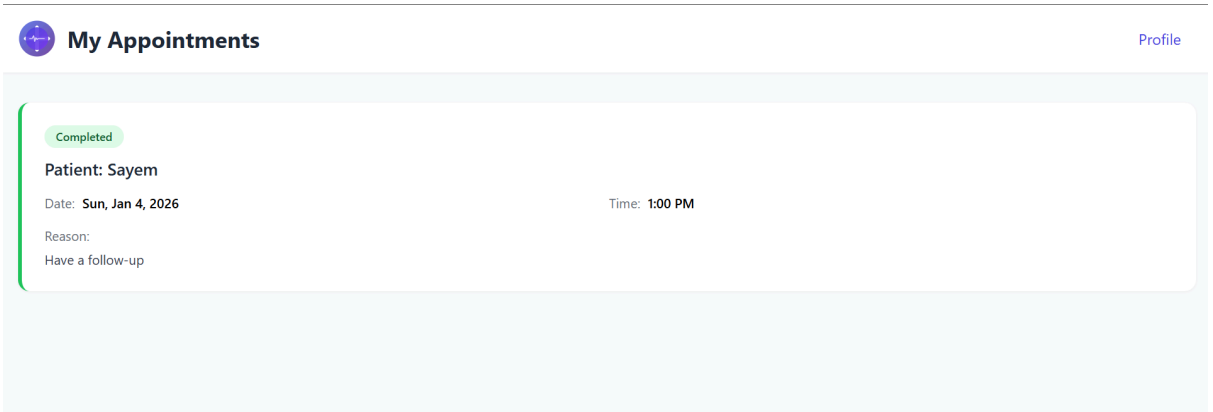


3. Doctor Interfaces:









4. Patient Interfaces:

SPHERE
Health Record System

My Appointments | My Profile

Our Doctors

Find the right specialist for your healthcare needs

Search by name or specialization...

Surgery (1 doctor)

Dr. Mehrab Sayem
Surgery
Book Appointment

Showing 1 of 1 doctors

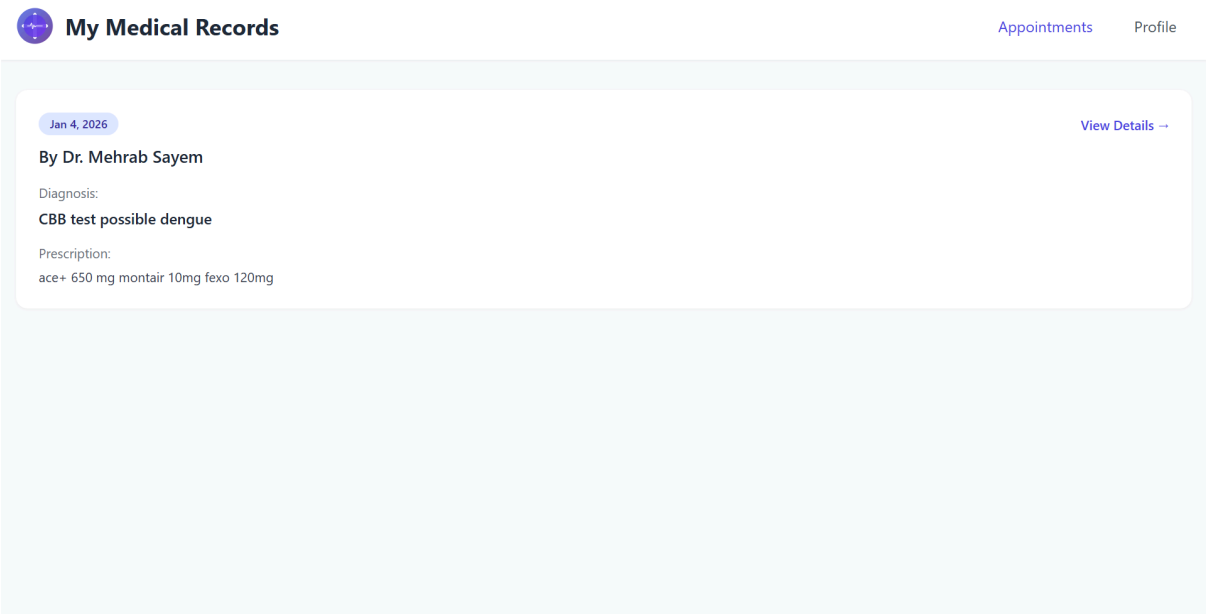
Dr. Mehrab Sayem
Surgery

Appointment Date *
dd-----yyyy

Preferred Time *
--:-- --

Reason for Visit *
Please describe your symptoms or reason for the appointment...

Your data is secure
All appointment details are encrypted using RSA and ECC algorithms. Data integrity is verified using HMAC.



14. Frontend Development

Snippet 1: Login with 2FA Detection

Handles user login and detects if 2FA verification is required before granting access.

```
const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  setError('');
  setLoading(true);

  try {
    const response = await authService.login(formData);

    if (response.requires_2fa) {
      navigate('/verify-2fa', { state: { tempToken: response.temp_token } });
    } else {
      navigate('/profile');
    }
  } catch (err: any) {
    setError(err.response?.data?.detail || 'Login failed');
  } finally {
    setLoading(false);
  }
};
```

Snippet 2: Registration with Encrypted Data Submission

It Submits user registration data to backend where it's automatically encrypted using RSA/ECC.

```
const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();

  if (formData.password !== formData.confirm_password) {
    setError('Passwords do not match');
    return;
  }
```

```
  try {
    if (registrationType === 'doctor') {
      await authService.registerDoctor({
        username: formData.username,
        email: formData.email,
        name: formData.name,
        specialization: formData.specialization,  // ECC encrypted
        contact_no: formData.contact_no,
        password: formData.password,
        confirm_password: formData.confirm_password
      });
    } else if (registrationType === 'patient') {
      await authService.registerPatient({
        username: formData.username,
        email: formData.email,
        name: formData.name,
        age: parseInt(formData.age),  // ECC encrypted
        sex: formData.sex,            // ECC encrypted
        contact_no: formData.contact_no,
        password: formData.password,
        confirm_password: formData.confirm_password
      });
    }

    navigate('/login');
  } catch (err: any) {
    setError(err.response?.data?.detail || 'Registration failed');
  }
};
```

Snippet 3: Two-Factor Authentication Verification
Verifies the 6-digit 2FA code sent to user's email before granting access.

```
const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  setLoading(true);

  try {
    await authService.verify2FA(tempToken, code);
    navigate('/profile');
  } catch (err: any) {
    setError(err.response?.data?.detail || 'Invalid verification code');
  } finally {
    setLoading(false);
  }
};

const handleResend = async () => {
  setResendLoading(true);
  try {
    await authService.resend2FA(tempToken);
    alert('New verification code sent to your email');
  } catch (err) {
    setError('Failed to resend code');
  } finally {
    setResendLoading(false);
  }
};
```

Snippet 4: Protected Routes with RBAC

Implements role-based access control on frontend routes, preventing unauthorized access.

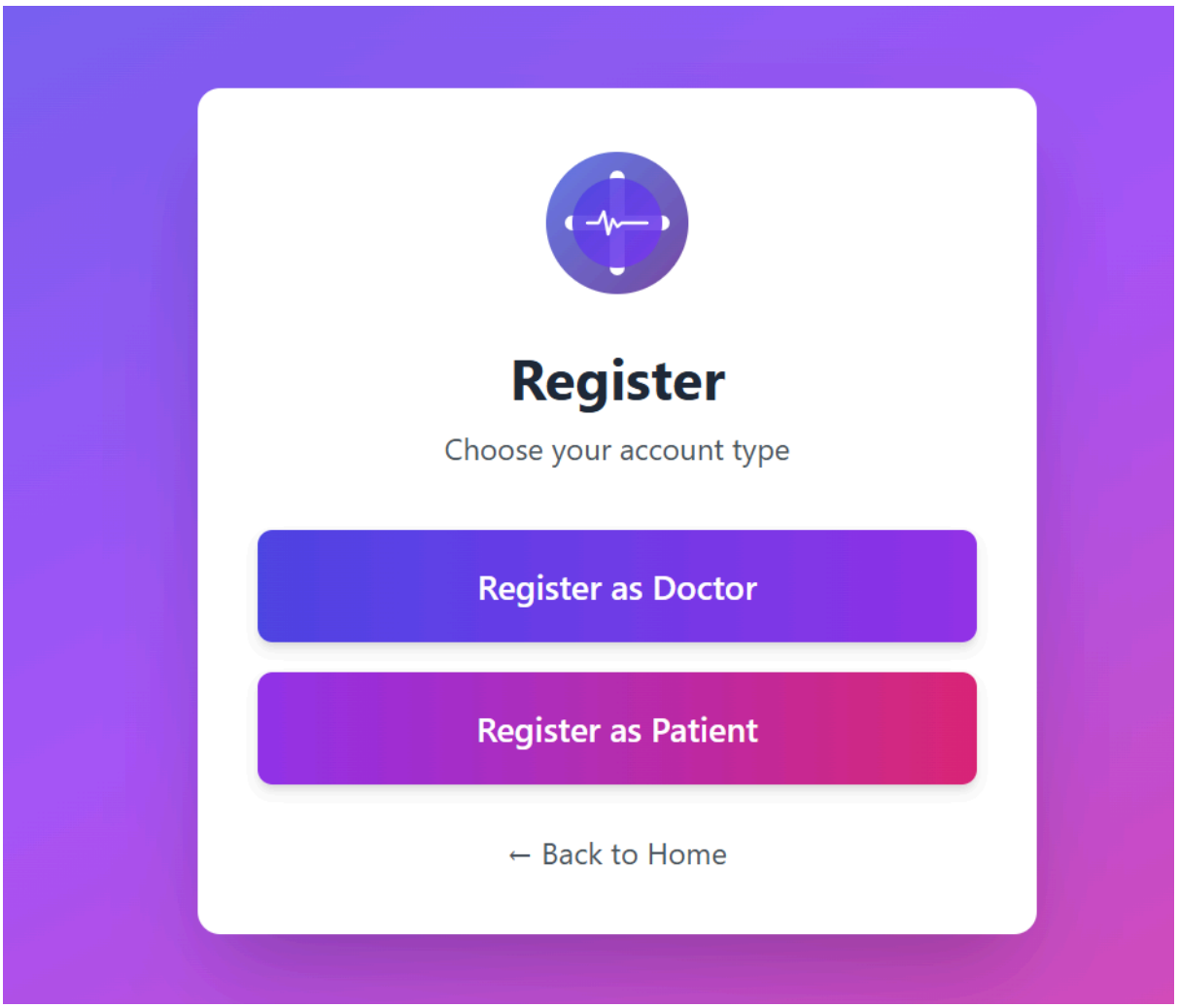
```
interface ProtectedRouteProps {
  children: React.ReactNode;
  requiredRole?: string;
}

const ProtectedRoute: React.FC<ProtectedRouteProps> = ({ children, requiredRole }) => {
  if (!isAuthenticated()) {
    return <Navigate to="/" replace />;
  }

  if (requiredRole) {
    const userRole = getUserRole();
    if (userRole !== requiredRole) {
      return <Navigate to="/profile" replace />;
    }
  }

  return <>{children}</>;
};
```

Snippet 5: Secure API Service with JWT Authentication
Configures Axios to automatically attach JWT bearer tokens to all API requests for secure communication.

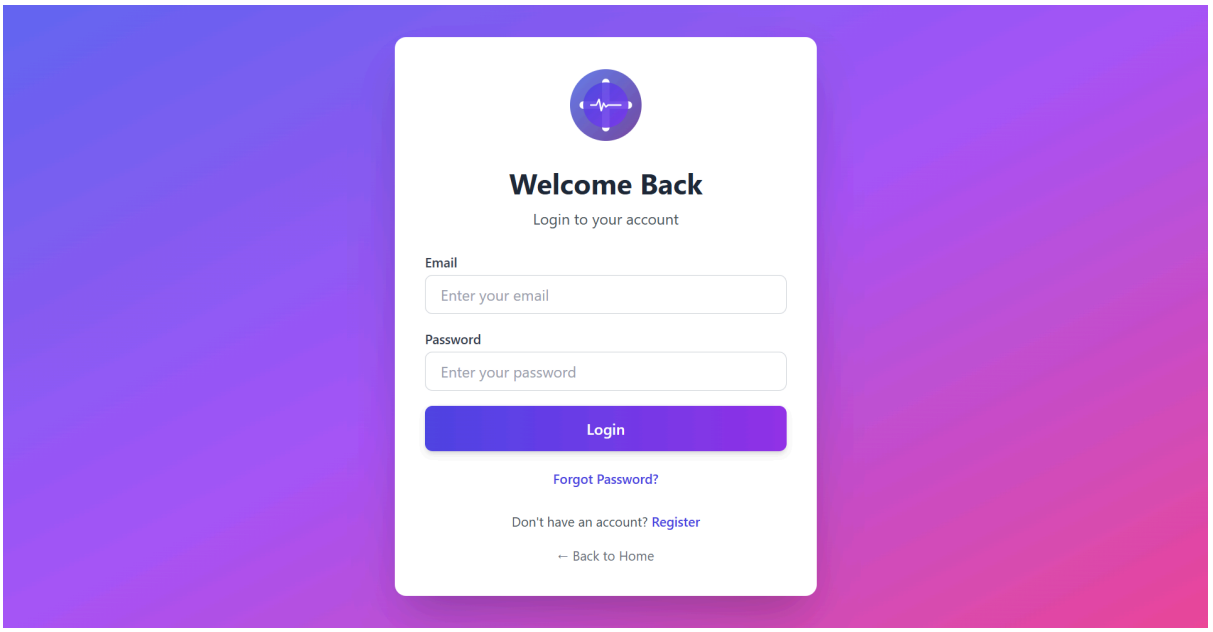
```
import axios from 'axios';

const api = axios.create({
  baseURL: '/api',
  headers: {
    'Content-Type': 'application/json',
  },
});

// Add JWT token to all requests
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

export default api;
```

15. User Manual

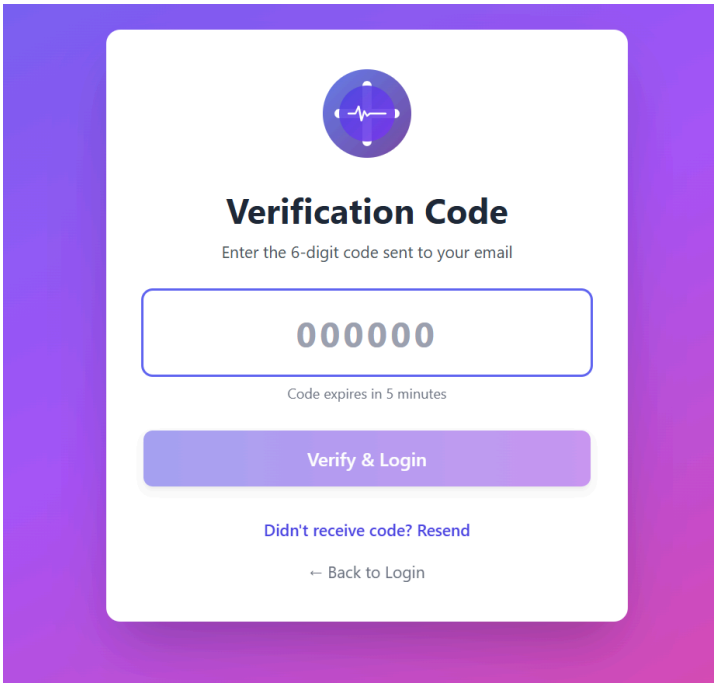
1. User registers based on their role by giving valid information



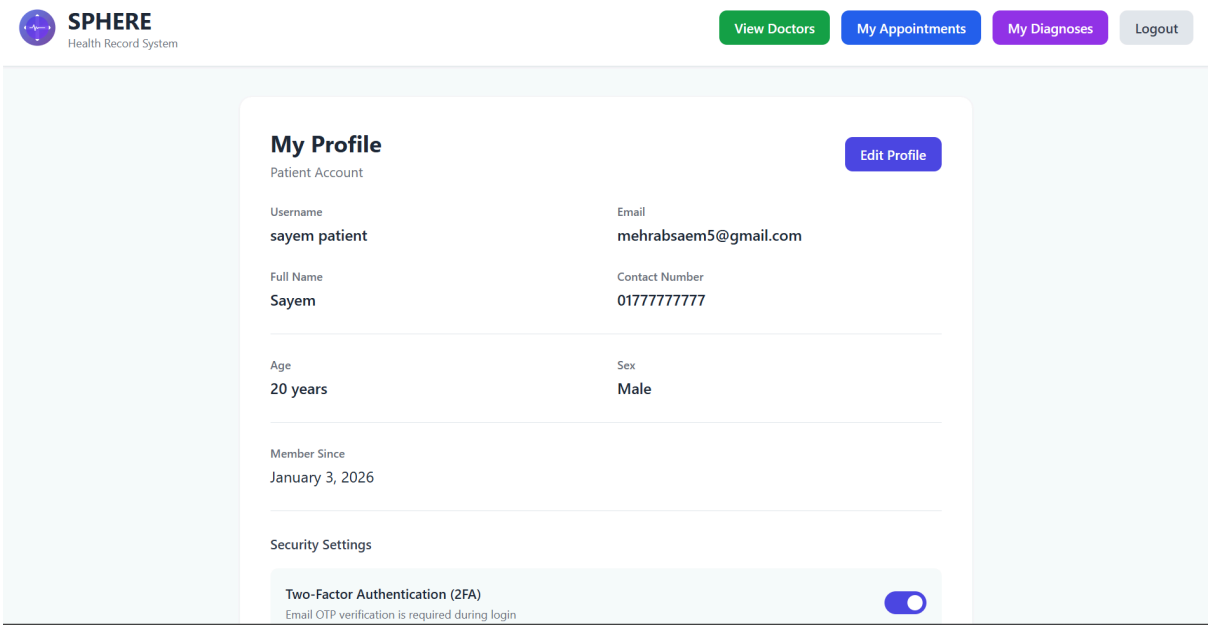
2. User based on their role (Doctor and Patient) Login Using valid email and password



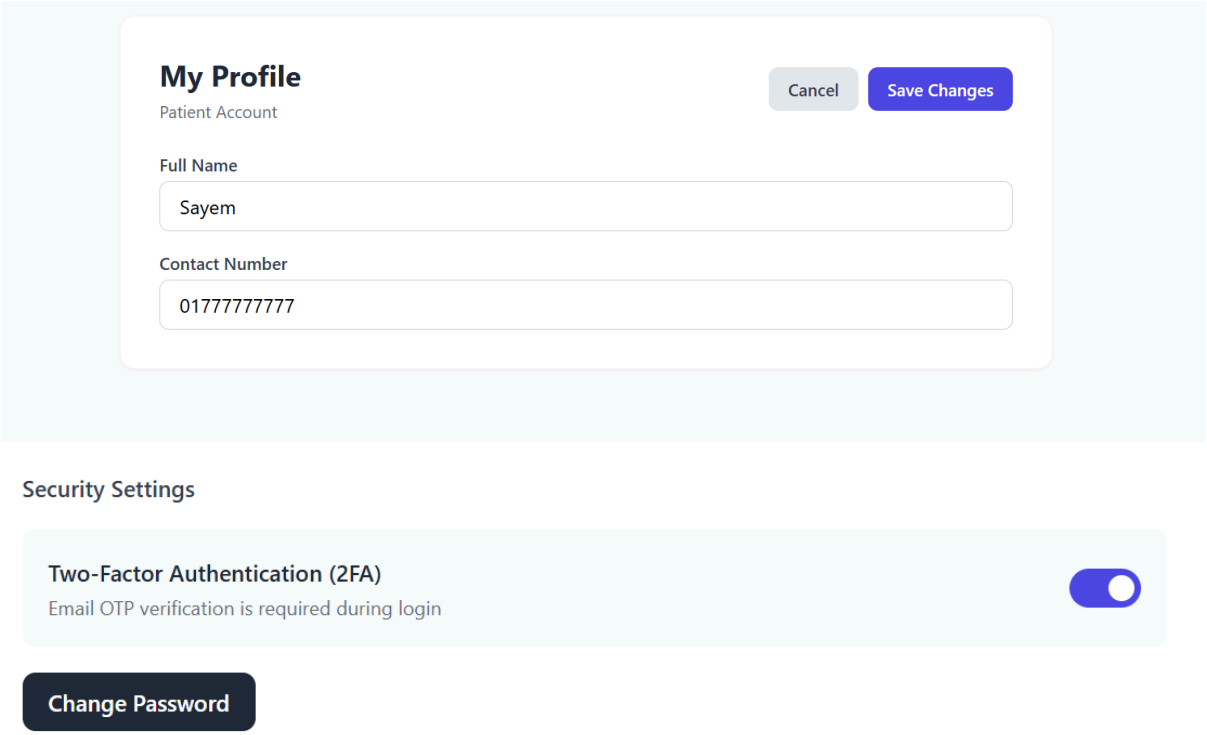
3. Enter the correct OTP sent to the user's registered email.



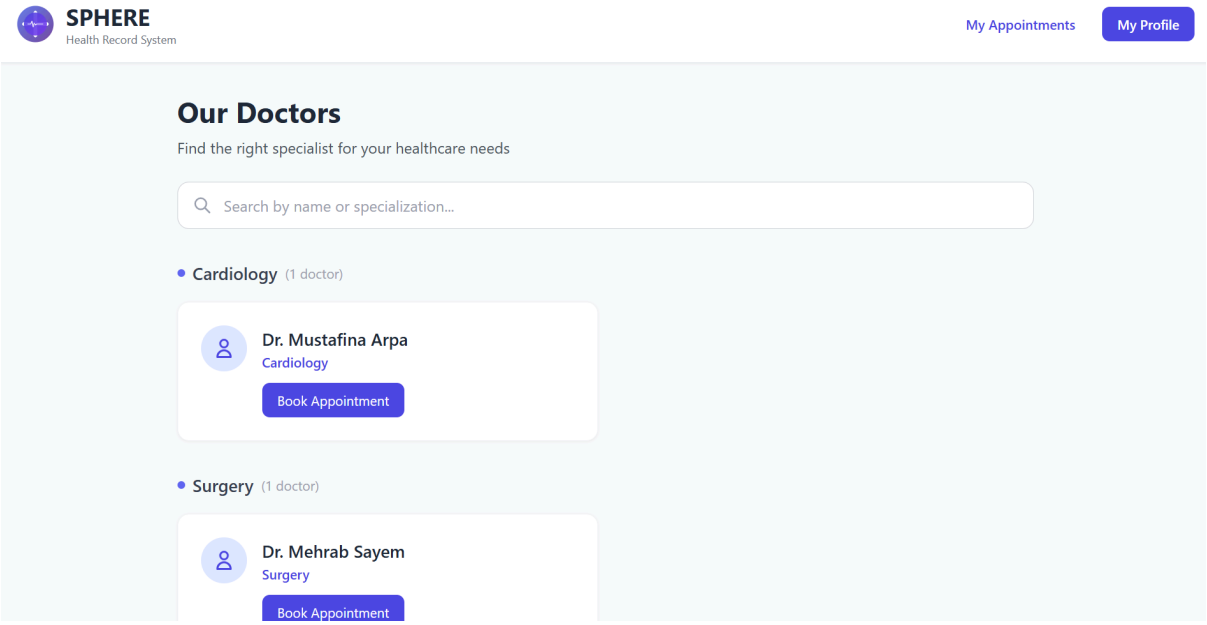
4. Upon Login user can see their profile and based on their role see options in the menu

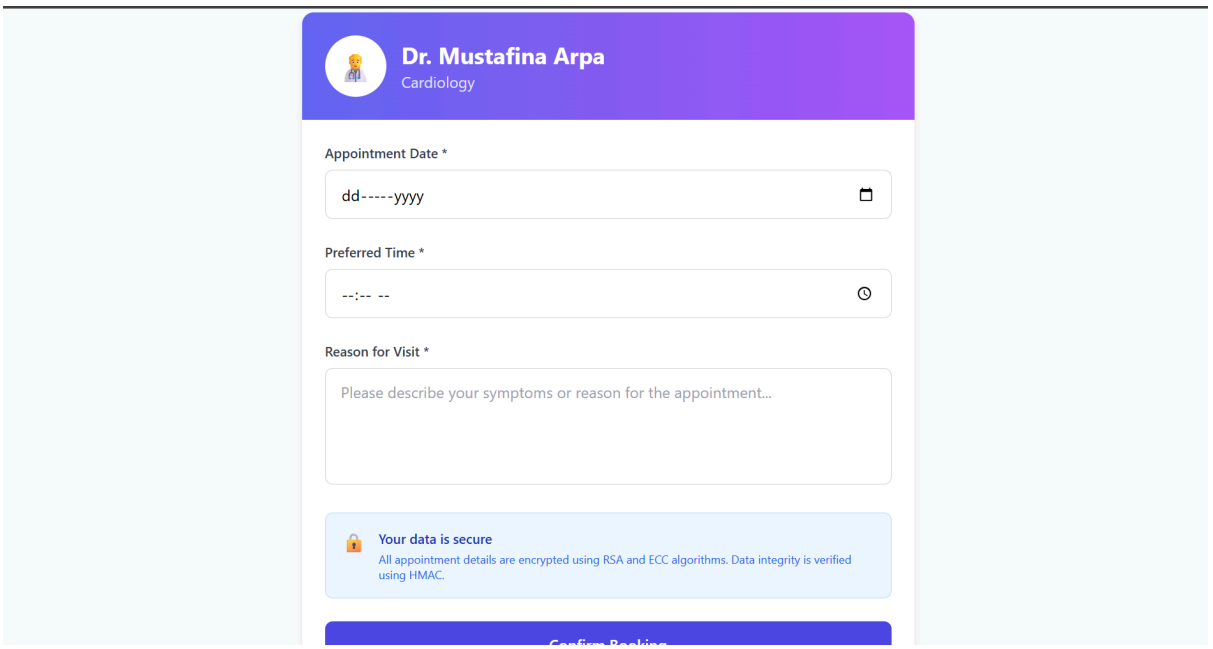


5. Usrer can edit their profile and turn on/off two factor authentication, change password

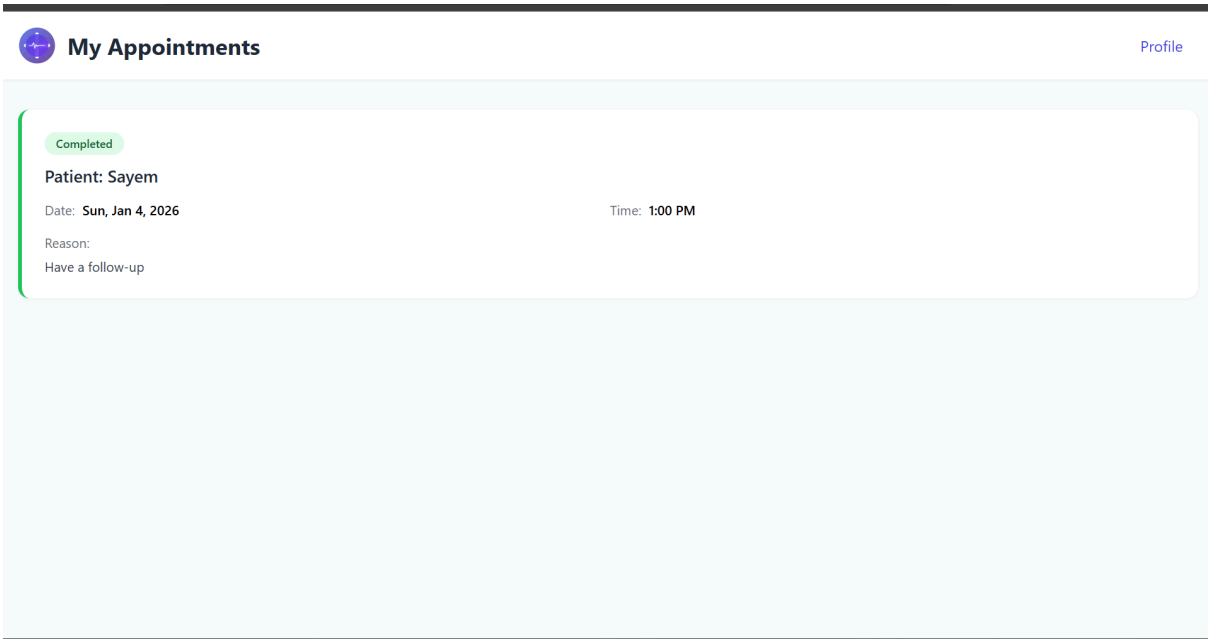


6. Patients can see doctor list and book an appointment of the doctor

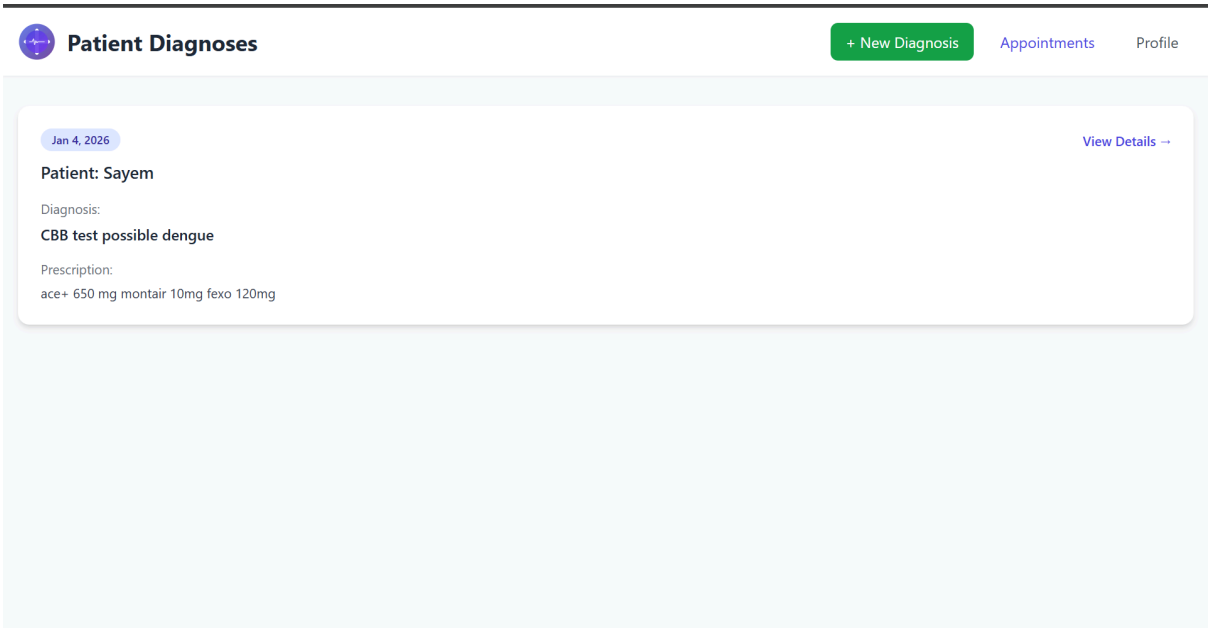




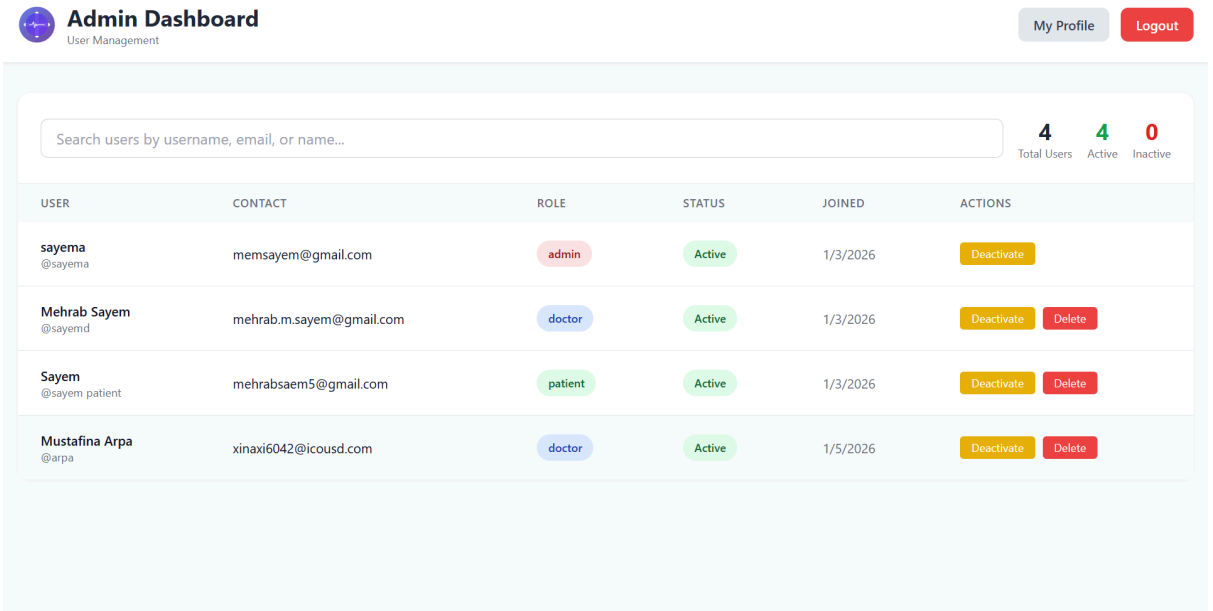
7. Doctors can see who booked appointment and confirm or cancel appointment or search any appointment



8. Doctors can add and see diagnosis of their patients



9. Admin can see user list and activate/deactivate or delete any user's account.



10. By clicking logout user can get out of their account.

16. Performance and Network Analysis

Generate Performance and Network analysis report using lighthouse DevTool. Add screenshots of the report. Also, attach a screenshot of your system in any mobile viewport. **The system UI must be responsive.**

17. Github Repo [Public] Link

https://github.com/mehrab-m-sayem/sphere-

18. Link of Deployed Project

19. Individual Contribution

Group member - 01	
Name: Mehrab Muhtasim Sayem	Student ID: 22101584
Functional Requirements which are developed by this member:	
1. User Authentication & Authorization	
2. Data Encryption & Decryption, Confirmation of Data Integrity. (partially)	
3. Medical Record Management	
4. Appointment Management	

Group member - 02	
Name: Mustafina Joarder	Student ID: 22101215
Functional Requirements which are developed by this member:	
1. Role Based user access	
2. Profile view (own + Admin can view everyone's profile)	
3. Profile Edit	
4. Data Encryption While Storing	

Group member - 03	
Name: Fuad Fardin	Student ID:22101027
Functional Requirements which are developed by this member:	
1. Key Management	
2. User Interface Requirements.	
3. Data Encryption & Decryption (partially)	
4.Confirmation of Data Integrity	

Group member - 04	
Name:	Student ID:
Functional Requirements which are developed by this member:	
1.	
2.	
3.	
4.	

20. References

a. https://www.teach.cs.toronto.edu/~csc110y/fall/notes/08-cryptography/05-rsa-cryptosystem-implementation.html
b. https://github.com/jchen2186/rsa-implementation
c. https://fastapi.tiangolo.com/
d. https://sqlite.org/docs.html
e. https://learn.microsoft.com/en-us/dotnet/api/system.security.cryptography.hmacsha256?view=net-10.0

f. https://en.wikipedia.org/wiki/Elliptic-curve_cryptography
g. https://tailwindcss.com/docs/installation/using-vite
h. https://react.dev/learn